

第 4 章

存储过程和触发器

朱泽明

4.0 变量的定义

www.qianrushi.com.cn

Mysql 变量：

1、 DECLARE variable_name datatype(size) DEFAULT default_value;
此处声明的相当于一个局部变量，在 end 之后便失效。
声明多个： DECLARE x, y INT DEFAULT 0

变量的定义

www.qianrushi.com.cn

2、 set @p=1;

此处的 session 变量不需要声明，mysql 会自动根据值类型来确定类型，这种变量要在变量名称前面加上“@”符号，叫做会话变量，代表整个会话过程他都是有作用的，这个有点类似于全局变量一样。这种变量用途比较广，因为只要在一个会话内（就是某个应用的一个连接过程中），这个变量可以在被调用的存储过程或者代码之间共享数据。

3、 变量赋值的方式，除了 set，还可以使用 selectinto.....的方式

两者的区别

www.qianrushi.com.cn

在调用存储过程时，以 DECLARE 声明的变量都会被初始化为 NULL。而会话变量（即 @ 开头的变量）则不会被再初始化，在一个会话内，只须初始化一次，之后在会话内都是对上一次计算的结果，就相当于在这个会话内的全局变量。

4.1 存储过程

www.qianrushi.com.cn

存储过程：为了方便现在和以后使用而保存起来的一条或者多条 MySQL 语句的集合

存储过程

www.qianrushi.com.cn

```
mysql> delimiter //  
mysql> create procedure test()  
-> begin  
-> select * from student limit 3;  
-> select * from course limit 2;  
-> end  
-> //
```

查询语句默认以分号作为结束。由于存储过程由多条语句组成，在写的过程中不能遇分号结束。

所以使用 `delimiter` 设置 `//` 作为 语句的结束标识。

存储过程

www.qianrushi.com.cn

```
mysql> delimiter ;  
mysql> call test();
```

stuId	stuName	stuAge	stuDepart	stuSex
10001	张三	NULL	数学系	男
10002	李四	32	化学系	男
10003	张紫柔	18	音乐系	女

3 rows in set (0.00 sec)

couId	couName	couCredit	tecId
30001	c++	4	20001
30002	java	3	20003

2 rows in set (0.00 sec)

存储过程

www.qianrushi.com.cn

删除存储过程

```
mysql> drop procedure test;  
Query OK, 0 rows affected (0.00 sec)
```


存储过程

www.qianrushi.com.cn

创建含有参数的存储过程—— out

```
mysql> create procedure  
-> proc_age(out minAge int,  
-> out maxAge int,  
-> out avgAge decimal(8, 2))  
-> begin  
-> select min(stuAge) from student into minAge;  
-> select max(stuAge) from student into maxAge;  
-> select avg(stuAge) from student into avgAge;  
-> end  
-> //
```

存储过程

www.qianrushi.com.cn

调用存储过程

```
mysql> delimiter ;  
mysql> call proc_age(@minAge, @maxAge, @avgAge);  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select @minAge, @maxAge, @avgAge;
```

+-----+	+-----+	+-----+
@minAge	@maxAge	@avgAge
+-----+	+-----+	+-----+
17	32	20.00
+-----+	+-----+	+-----+

传入的三个参数不需要提前定义好，在调用存储过程是直接写上就可以

存储过程

www.qianrushi.com.cn

创建含有参数的存储过程—— in

```
mysql> delimiter //
mysql> create procedure
-> proc_salary(in id int, out salary decimal(8,2))
-> begin
-> select tecSalary from teacher where tecId = id
-> into salary;
-> if salary > 5000 then
-> select salary * 0.9 into salary;
-> end if;
-> end //
```

存储过程

www.qianrushi.com.cn

```
mysql> call proc_salary(20004, @salary);  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select @salary;
```

```
+-----+  
| @salary |  
+-----+  
| 18000.00 |  
+-----+
```

存储过程

www.qianrushi.com.cn

```
mysql> show create procedure proc_salary \G;
***** 1. row *****
      Procedure: proc_salary
      sql_mode:
      Create Procedure: CREATE DEFINER=`root`@`localhost` PROCEDURE `proc_salary`
      `(in id int, out salary decimal(8,2))
begin
select tecSalary from teacher where tecId = id
into salary;
if salary > 5000 then
select salary * 0.9 into salary;
end if;
end
character_set_client: utf8
collation_connection: utf8_general_ci
      Database Collation: latin1_swedish_ci
1 row in set (0.00 sec)
```

IF 分支选择语句

www.qianrushi.com.cn

```
IF condition THEN  
ELSEIF condition THEN  
END IF;
```

例子：

delimiter //

```
create procedure lianxi1(in a int,in b int)  
begin  
IF a>b THEN select 'a>b';  
ELSEIF b>a THEN select 'b>a';  
end if;  
end //
```

CASE 分支选择语句

www.qianrushi.com.cn

CASE xxx

WHEN x THEN sql_statement;

WHEN y THEN sql_statement;

WHEN z THEN sql_statement;

ELSE sql_statement;

END CASE;

例子 : delimiter //

create procedure lianxi2(in a int)

begin

CASE a

WHEN 10 THEN select 'a=10';

WHEN 100 THEN select 'a=100';

ELSE select 'a=???';

END CASE;

end //

WHILE 循环语句

www.qianrushi.com.cn

```
WHILE xxx DO  
sql_statement;  
...  
END WHILE;
```

或者：

```
[label:] WHILE expression DO  
  
statements  
  
END WHILE [label];
```


WHILE 循环语句

www.qianrushi.com.cn

例子 : delimiter //

```
create procedure lianxi3(in a int)
```

```
begin
```

```
WHILE a>0 DO
```

```
select 'a = ',a;SET a=a-1;
```

```
END WHILE;
```

```
End //
```

或者 : delimiter //

```
create procedure lianxi4(in a int)
```

```
begin
```

```
XYZ:WHILE a>0 DO
```

```
select 'a = ',a;SET a=a-1;
```

```
END WHILE XYZ;
```

```
end //
```

REPEAT 循环语句

www.qianrushi.com.cn

```
REPEAT  
sql_statement;  
...  
UNTIL xxx END REPEAT;
```

或者：

```
[label:] REPEAT  
  
statements  
  
UNTIL expression END REPEAT [label];
```

REPEAT 循环语句

www.qianrushi.com.cn

注释：这是一个 REPEAT 循环的例子，功能和 WHILE 循环一样。

区别在于它在执行后检查结果，而 WHILE 则是执行前检查，可能等同于 DO WHILE 吧

例子：

```
delimiter //
```

```
create procedure lianxi5(in a int)
```

```
begin
```

```
repeat
```

```
select 'a = ',a;SET a=a-1;
```

```
until a<0 end repeat;
```

```
End //
```

第十八章 存储过程

www.qianrushi.com.cn

小结

4.3 触发器

www.qianrushi.com.cn

触发器是一个特殊的存储过程，不同的是存储过程要用 CALL 来调用，而触发器不需要使用 CALL，也不需要手工启动，只要当一个预定义的事件发生的时候，就会被 MYSQL 自动调用。

触发器

www.qianrushi.com.cn

创建触发器

语法如下：

```
CREATE TRIGGER trigger_name trigger_time trigger_event  
ON tbl_name FOR EACH ROW trigger_stmt
```

触发程序是与表有关的命名数据库对象，当表上出现特定事件时，将激活该对象。

触发器

www.qianrushishi.com.cn

触发程序与命名为 `tbl_name` 的表相关。 `tbl_name` 必须引用永久性表。不能将触发程序与临时表或视图关联起来。

`trigger_time` 是触发程序的动作时间。它可以是 `BEFORE` 或 `AFTER`，以指明触发程序是在激活它的语句之前或之后触发。

`trigger_event` 指明了激活触发程序的语句的类型。 `trigger_event` 可以是下述值之一：

- `INSERT`：将新行插入表时激活触发程序，例如，通过 `INSERT`, `LOAD DATA` 和 `REPLACE` 语句。
- `UPDATE`：更改某一行时激活触发程序，例如，通过 `UPDATE` 语句。
- `DELETE`：从表中删除某一行时激活触发程序，例如，通过 `DELETE` 和 `REPLACE` 语句。

触发器

www.qianrushi.com.cn

对于具有相同触发程序动作时间和事件的给定表，不能有两个触发程序。例如，对于某一表，不能有两个 BEFORE UPDATE 触发程序。

但可以有 1 个 BEFORE UPDATE 触发程序和 1 个 BEFORE INSERT 触发程序，或 1 个 BEFORE UPDATE 触发程序和 1 个 AFTER UPDATE 触发程序。

trigger_stmt 是当触发程序激活时执行的语句。

如果你打算执行多个语句，可使用 BEGIN ... END 复合语句结构。这样，就能使用存储子程序中允许的相同语句。

触发器

www.qianrushi.com.cn

创建一个单执行语句的触发器

```
CREATE TABLE account(acct_num INT ,amount  
DECIMAL(10,2));
```

```
CREATE TRIGGER ins_sum BEFORE INSERT ON account  
FOR EACH ROW SET @SUM=@SUM+new.amount;
```

首先创建一个 account 表，表中有两个字段，分别为：
： acct_num 字段（定义为 int 类型）
amount 字段（定义成浮点类型）；其次创建一个名为 ins_sum 的触发器，触发的条件是向数据表 account 插入数据之前，对新插入的 amount 字段值进行求和计算

触发器

www.qianrushi.com.cn

```
DECLARE @num INT
```

```
SET @num=0
```

```
INSERT INTO account VALUES(1,1.00),(2,2.00)
```

```
SELECT @num
```

首先创建一个 account 表，在向表 account 插入数据之前，计算所有新插入的 account 表的 amount 值之和，

触发器的名称为 ins_num，条件是在向表插入数据之前触发。

触发器

www.qianrushi.com.cn

多个执行语句

delimiter &&

...body

&&

delimiter ;

触发器

www.qianrushi.com.cn

"NEW . column_name" 或者 "OLD . column_name". 这样在技术上处理 (NEW | OLD . column_name) 新和旧的列名属于创建了过渡变量 ("transition variables") 。对于 INSERT 语句, 只有 NEW 是合法的; 对于 DELETE 语句, 只有 OLD 才合法; 而 UPDATE 语句可以在和 NEW 以及 OLD 同时使用。

触发器

www.qianrushi.com.cn

下面是一个 UPDATE 中同时使用 NEW 和 OLD 的例子

。

[sql] view plain copy

```
CREATE TRIGGER up  
BEFORE UPDATE ON student  
FOR EACH ROW  
BEGIN  
SET @old = OLD.age;  
SET @new = NEW.age;  
END;
```

现在如果 student 表中的 age 列的值是 20，那么执行了 "UPDATE age SET age=22" 之后 @old 的值会变成 20，而 @new 的值将会变成 22。

触发器

www.qianrushi.com.cn

查看触发器

查看触发器是指数据库中已存在的触发器的定义、状态、语法信息等。可以使用 SHOW TRIGGERS 和在 TRIGGERS 表中查看触发器信息

SHOW TRIGGERS

```
SELECT * FROM `information_schema`.`TRIGGERS` WHERE  
`TRIGGER_NAME`='ins_sum'
```

触发器

www.qianrushi.com.cn

删除触发器

使用 DROP TRIGGER 语句可以删除 MYSQL 中已经定义的触发器，删除触发器的基本语法

DROP TRIGGER [schema_name.]trigger_name

其中（ schema_name ）是可选的

如果省略了 schema（方案），将从当前方案中舍弃触发程序。

删除 ins_sum 触发器

DROP TRIGGER `school`.`ins\_sum`

触发器 ins_sum 删除成功

触发器

www.qianrushi.com.cn

总结

对于相同的表，相同的事件只能创建一个触发器，
比如对表 account 创建了 BEFORE INSERT 触发器

那么如果对表 account 再次创建一个 BEFORE INSERT
触发器，MYSQL 就会报错，此时，只可以在表
account 上

创建 AFTER INSERT 或者 BEFORE UPDATE 类型的触发
器